Auditing a Volume Confound in Proxy-Label Supply Chain Stress Prediction: A Validation-at-Scale Case Study on M5

Suhail Yunus[1]

[1]Independent Researcher

Abstract

Public retail data rarely record stockouts or unfulfilled orders, so supervised risk models often use proxy labels. We study an M5-derived label for next-day "demand stress" and show why validation of the label is distinct from validation of the classifier. Two temporal leakage defects in an early pipeline were caught by mechanism-level regression tests even though holdout metrics appeared plausible. More importantly, the original 90th-percentile threshold pooled each item across ten stores. Under that definition, store-level stress rate was strongly associated with store sales volume ($r = 0.853$, $p = 0.0017$; $n = 10$ stores), and the association remained within item across 30,490 item-store series ($r = 0.369$). A series-normalized item-store threshold, which matches our intended estimand of demand unusual for a location's own history, reduced these correlations to $r = 0.043$ and $r = -0.060$, respectively. This is an estimand change, not a universally correct label for every supply-chain decision. We then audit class-weighted XGBoost scores using chronologically disjoint training, calibration, and final-evaluation periods. Isotonic calibration improved Brier score from 0.2351 to 0.0812 on 1,000 series and from 0.2274 to 0.1053 on the full catalog. Against the appropriate prevalence-only references, however, the calibrated Brier Skill Scores were only 0.039 and 0.048. Calibration therefore repaired severe overconfidence but delivered modest probabilistic skill. Under stated constant costs, thresholds selected before final evaluation produced similar, though not identical, outcomes before and after calibration. A model-family robustness check confirms these conclusions are not an artifact of the specific gradient-boosting library used. The case study demonstrates that proxy-label grouping, temporal independence, probability meaning, and operating-policy selection require separate tests.

1 Introduction

Predicting operational risk in a supply chain – which items, at which locations, are likely to face unusual demand pressure – is a problem most public retail datasets cannot answer directly, because they do not record the outcome that actually matters: whether a stockout, a supplier delay, or an unfulfilled order occurred. Practitioners building such models are therefore routinely forced into *proxy-label* modeling: defining a measurable stand-in for the unobserved outcome (here, unusually high demand relative to a series' own history) and treating it as ground truth for supervised learning.

Proxy labels carry a risk that ordinary machine learning validation practice does not automatically catch. A model can achieve strong, stable holdout performance while the label itself encodes an artifact of how it was constructed rather than the phenomenon it is meant to stand in for. Standard train/test splits, cross-validation, and even chronological holdout evaluation – the discipline this paper otherwise argues for – are silent on whether the *label definition* itself is sound, because

every one of those procedures evaluates a model's ability to predict the label as given, not whether the label means what it is assumed to mean.

This paper documents a case where exactly that happened. Using the M5 Forecasting – Accuracy dataset [Makridakis et al., 2022], we define a stress label as sales exceeding a historical 90th percentile and predict that event one day ahead. An early version pooled the percentile across stores. It passed chronological evaluation but encoded store volume in a target intended to represent demand unusual relative to each location's own history.

We make three contributions, each in the form of a tested claim rather than an assumed one:

1. We document two chronological data-leakage defects found during initial development – a stress threshold estimated over each series' complete history rather than its training-period history alone, and a rolling-window feature that included the current day's own value – and show that standard holdout evaluation did not surface either one; both were caught only by explicit regression tests asserting temporal independence (Section 4).

2. We quantify the store-volume association both across stores and within item, then replace the pooled threshold with a series-normalized definition aligned with our stated estimand (Sections 5 and 6).

3. We evaluate calibration against the correct prevalence-only Brier reference and keep calibrator fitting and final reporting chronologically disjoint. We also separate retrospective oracle thresholds from thresholds selected before final evaluation (Section 7), and confirm the resulting conclusions are not an artifact of the specific gradient-boosting library used (Section 8).

The contribution is not a new algorithm or a stockout model. It is a documented validation methodology for a proxy-label, class-imbalanced, cost-sensitive prediction problem, including corrections to claims that appeared convincing before the relevant assumptions were tested.

2 Related Work

Benchmark dataset. The M5 Forecasting competition [Makridakis et al., 2022] evaluated point-forecast accuracy on 42,840 hierarchical unit-sales series from Walmart; we use its constituent sales, calendar, and pricing files but repurpose the task from demand forecasting to binary stress classification, a proxy-label reframing not evaluated in the original competition.

Data leakage. Kaufman et al. [2012] formalize leakage in predictive modeling as the introduction of information about the target that would not legitimately be available at prediction time, and argue that a strict *learn-predict separation* is the primary defense. The two leakage defects we report (Section 4) are concrete instances of exactly this failure mode in a time-series setting, where the temporal ordering of the data is the specific axis along which separation must be enforced.

Demand classification. Syntetos et al. [2005] propose classifying demand series by average inter-demand interval and the squared coefficient of variation of demand sizes, yielding the smooth/erratic/intermittent/lumpy taxonomy we use to design a stratified sample (Section 6) that deliberately oversamples the harder, more business-relevant intermittent and lumpy segments rather than sampling uniformly at random.

Gradient boosting. We use XGBoost [Chen and Guestrin, 2016] as the primary classifier. Model-family novelty is not claimed; a logistic regression baseline is included to contextualize probabilistic performance, and Section 8 compares against LightGBM [Ke et al., 2017] and CatBoost [Prokhorenkova et al., 2018].

Probability calibration. Niculescu-Mizil and Caruana [2005] show that boosted trees systematically distort predicted probabilities away from 0 and 1 in a characteristic sigmoid-shaped pattern, and evaluate Platt scaling [Platt, 1999] and isotonic regression [Zadrozny and Elkan, 2002] as corrections. Our finding (Section 7) that a boosted-tree model trained with class-imbalance reweighting is severely miscalibrated is consistent with this literature; our contribution is showing, on a real cost-sensitive decision problem, that the correction changes what a score *means* far more than it changes which decision the score implies.

3 Dataset and Proxy Target Definition

M5 provides daily unit sales for 3,049 items across 10 stores in 3 U.S. states (30,490 item-store series in total), spanning 1,913 days, together with a calendar file (weekday, month, year, event flags, SNAP eligibility) and a weekly item-store price file.

No inventory, stockout, or supplier-delay signal exists anywhere in M5. We therefore define a proxy target:

$$\text{stress\_event}_{i,s,t} = \mathbb{1}\left[\text{sales}_{i,s,t} > Q_{0.90}\big(\text{sales}_{i,s,\cdot} \mid t' \le t_{\text{split}}\big)\right] \tag{1}$$

for item i, store s, and day t. The model uses information available through day $t-1$ to predict whether day t exceeds the fixed threshold. The quantile is estimated only from days on or before t_{split} and then applied to later periods. Because unit sales are discrete and often zero, the strict $>$ rule does not force the realized event rate to equal 10%.

The 80/20 chronological train/holdout split day is derived once from the full 1,913-day range and is scale-invariant: because every series shares an identical day range, the split day computed on a 100-item sample is identical to the split day computed on the full 30,490-series catalog, which we confirmed empirically holds exactly (day 1,531) across every sample size tested.

4 Leakage-Safe Pipeline Design

Two chronological leakage defects were identified and fixed during initial development. Neither was surfaced by plausible-looking aggregate holdout performance; each required a test of the leakage mechanism.

Threshold estimation window. An early implementation estimated the 90th-percentile stress threshold (Section 3) over each series' complete history, including holdout-period days. This is leakage under Kaufman et al. [2012]'s formulation: the label applied to a training-period row was informed, in part, by sales that occur after that row in time. The fix constrains threshold estimation to training-period days only, as stated in Equation 1.

Rolling-feature window. The 7-day rolling mean and standard deviation features were initially computed as a trailing window inclusive of the current day, so that a demand spike would appear in its own rolling-average feature. The fix shifts the window by one day before applying it (`sales.shift(1).rolling(7)` in the pandas implementation; `ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING` in the equivalent SQL), so that day t's features never include day t's own sales value.

Both defects were fixed prior to any of the results in Section 5 onward, and both are now guarded by regression tests that construct a synthetic series with a known, hand-computable spike and assert the exact numeric value of the resulting feature – a test strong enough to fail immediately if either defect were reintroduced.

5 The Store-Volume Confound

5.1 Discovery

The original implementation grouped the 90th-percentile threshold by item alone, pooling the ten stores carrying each item. The concern was recorded as an open limitation and was later raised independently during external code review, prompting a direct test.

The mechanism is direct: pooling sales across stores means a single threshold is applied to every store carrying that item, regardless of each store's individual sales volume. A store that sells more units of an item on average will exceed that pooled threshold more often, independent of whether its demand is genuinely more volatile relative to its own baseline.

5.2 Verification

We computed each store's holdout stress rate and average daily sales. Because ten aggregated observations provide limited precision, we report a p-value and Fisher interval and supplement the store-level result with a within-item analysis over all 30,490 item-store series.

Table 1: Association between sales volume and holdout stress rate under two proxy definitions.

	Item-only threshold	Item-store threshold
Stress-rate range across 10 stores	7.7%–19.9%	10.3%–16.1%
Store-level Pearson r ($n = 10$)	0.853	0.043
Two-sided p-value	0.0017	0.905
Fisher 95% interval	[0.483, 0.965]	[−0.603, 0.655]
Within-item Pearson r ($n = 30{,}490$)	0.369	−0.060

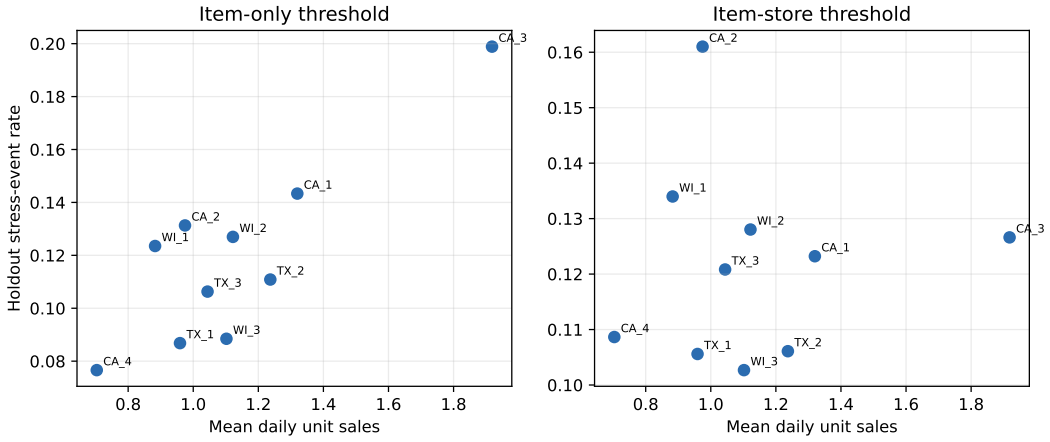

Figure 1: Store-level volume and holdout stress rate. The ten-store aggregation is descriptive; the within-item analysis in Table 1 provides the higher-resolution check.

5.3 Estimand-aligned revision

We revised the grouping to (item, store), giving each series a threshold estimated from its own training history. This aligns with the intended estimand: demand unusual for that item at that

location. It should not be interpreted as universally "correct." An item-level pooled threshold could be appropriate for a different question, such as identifying absolute network pressure. The evidence establishes strong volume contamination relative to our intended series-normalized estimand; it does not establish that store volume is causally irrelevant to supply-chain risk.

5.4 A second defect surfaced by the fix

Re-validating the corrected label at full catalog scale required processing the 58.3-million-row long-format dataset in per-store chunks to fit within available memory (Section 6), since the full dataset does not fit in memory on typical development hardware. This chunking approach silently introduced a second defect: one-hot encoding the `store_id` and `state_id` location features against *only the categories present in the current chunk* produces zero columns when a chunk contains a single store, since one-hot encoding a column with exactly one observed category and `drop_first=True` has nothing left to encode. This failure is silent – no exception is raised, no warning is emitted – and a joint model trained across all chunks would have been trained with no location information whatsoever, despite location features appearing to be present in the pipeline's code path. We caught this by asserting that every chunk's output schema was identical before trusting any downstream result, and fixed it by encoding against a fixed, externally-supplied list of all 10 stores and 3 states, so that every chunk produces the same, stackable set of columns regardless of which single store it contains. This defect is unrelated to the store-volume confound but was found in the course of investigating it, and is reported here because it exemplifies the same general risk: a validated-looking pipeline stage that is silently wrong in a way ordinary execution does not reveal.

6 Validation at Scale

6.1 Stratified sampling design

Beyond a 100-item development sample (equivalent to 1,000 series, since every item spans all 10 stores), we constructed a stratified 5,000-series sample by drawing 500 items stratified by department and by demand-pattern class, using the Syntetos et al. [2005] taxonomy computed from each item's average inter-demand interval and the squared coefficient of variation of its non-zero demand sizes. Intermittent and lumpy items – the more business-relevant and empirically harder segment – were oversampled by a factor of 1.5 relative to their natural catalog share (44% → 53% intermittent representation in the sample). Because every item spans all 10 stores, sampling 500 items yields exactly 5,000 series with no store-representation bias.

6.2 Full-catalog processing

The complete catalog (30,490 series, 58.3 million long-format rows) does not fit in memory on the reference development hardware used for this work (a 3.9 GB, single-core container). We processed it store-by-store (10 chunks of approximately 5.8 million rows each), checkpointing each chunk to disk before processing the next, so that an interruption loses at most one chunk of work rather than the entire run. For the business-impact results reported in this section, the model was trained on a 20% row-subsample of the training period for computational tractability; the holdout evaluation itself was never subsampled, since the final business-impact figures are the numbers that matter most and should not be the ones approximated.

6.3 Results under the series-normalized definition

For a stated decision illustration, a missed event costs \$20, a false alert costs \$12, and reviewing a true alert costs \$10. Relative to taking no action, the incremental net is

$$\text{net} = (20 - 10)\,TP - 12\,FP. \tag{2}$$

These values are assumptions, not observed business costs. Table 2 reports both a fixed raw-score threshold and a retrospective oracle threshold chosen on the same holdout. The latter is an upper-bound diagnostic, not an unbiased deployment estimate.

Table 2: Illustrative business outcomes under the series-normalized label. Oracle values select and evaluate a raw-score threshold on the same holdout. Net per million rows makes scales comparable.

Scale	Rows	Prev.	Net@.80	Net@.80/M	Oracle (τ, net)	Oracle/M
5,000 (seed 42)	1,903,223	.134	−\$9,078	−\$4,770	(.89, \$27,584)	\$14,493
5,000 (seed 17)	1,903,760	.132	\$1,284	\$674	(.90, \$43,952)	\$23,087
5,000 (seed 123)	1,905,235	.136	\$22,980	\$12,062	(.83, \$41,258)	\$21,655
30,490 (full)	11,607,375	.122	−\$209,794	−\$18,074	(.90, \$47,148)	\$4,062

The corrected three-seed runs overturn an earlier claim that threshold 0.80 consistently loses money: its sign changes across samples. Oracle net also varies from \$27,584 to \$43,952. The full-catalog oracle remains positive, but its normalized value is much smaller. These results support ranking the review queue under the assumed costs; they do not support a stable dollar forecast.

7 Probability Calibration

7.1 Motivation and methodology

Every threshold in Section 6 is a raw XGBoost score. Class weighting changes the objective being optimized, so a raw score of 0.80 should not be read as an 80% event probability without an empirical calibration check.

We test this directly rather than continue to assert it as an untested limitation, using a three-way chronological split that separates model training, calibrator fitting, and final reporting into three disjoint, chronologically ordered periods:

$$\text{train } (\text{day} \le t_{\text{split}}) \;\rightarrow\; \text{calibration } (t_{\text{split}} < \text{day} \le t_{\text{calib}}) \;\rightarrow\; \text{final\_eval } (\text{day} > t_{\text{calib}}) \tag{3}$$

The first 40% of holdout days fit an isotonic calibrator [Zadrozny and Elkan, 2002]; the later 60% are used once for the results below. `FrozenEstimator` wraps the trained base model so calibration never refits it. For operating-policy evaluation, the threshold is selected on the calibration period and then fixed before final evaluation. We also show an oracle selected on final evaluation only to quantify the attainable upper bound.

7.2 Results

For a final-period prevalence p, the Brier score of a prevalence-only predictor is $p(1-p)$, not 0.25 unless $p = 0.5$. We therefore report both that reference and Brier Skill Score, $1 - BS_{\text{model}}/BS_{\text{reference}}$.

Table 3: Final-evaluation probability quality. AP baseline equals prevalence; positive Brier Skill Score indicates improvement over a constant prevalence forecast.

Scale	Prev.	Ref. BS	Raw BS	Iso. BS	Iso. BSS	AP raw/iso.
1,000 series	.0931	.0844	.2351	.0812	.0389	.1842/.1796
30,490 series	.1266	.1106	.2274	.1053	.0475	.2372/.2357

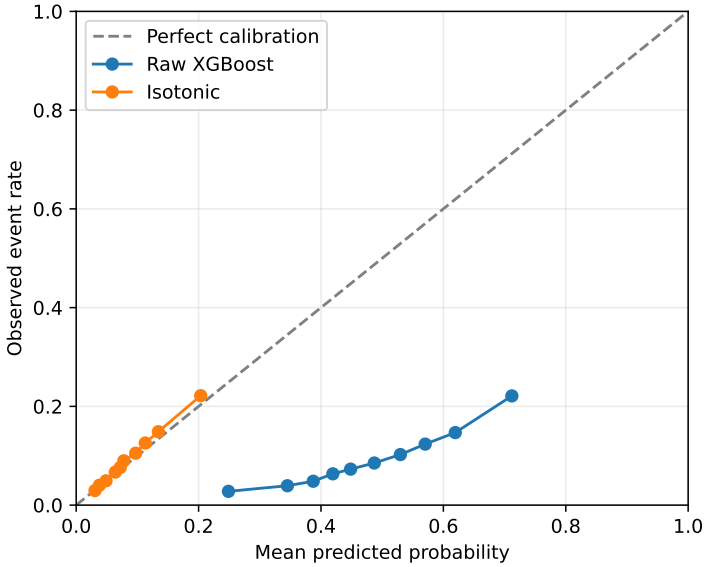

Figure 2: Quantile-binned reliability diagram, 1,000-series experiment, final-evaluation period only.

Calibration produces a large absolute Brier improvement, but the correct reference changes its interpretation: probabilistic skill above prevalence is approximately 4–5%. The unweighted logistic baseline at 1,000-series scale has AP 0.1333 and Brier score 0.0841, showing that XGBoost provides better ranking while calibration is needed for probability quality.

7.3 Why calibration barely changes the business decision

Platt scaling is strictly monotone, while isotonic regression is non-decreasing and can collapse observations into tied plateaus. Neither reverses an ordering, so a threshold selected at a given rank position on the calibration period selects the identical set of final-evaluation observations whether it is expressed in raw or calibrated units.

At full scale, the raw threshold selected on the calibration period was 0.9041 and produced \$28,806 on final evaluation; the corresponding isotonic threshold was 0.5498. Because threshold selection here operates on rank position and isotonic calibration preserves rank order exactly, this isotonic threshold selects the identical alert set (6,922 alerts; 5,085 true positives; 1,837 false positives) and therefore the identical \$28,806 net – the ex-ante policy value is not merely similar but mathematically identical under a single global threshold and constant per-case costs. The two thresholds' *retrospective* final-period oracle values, by contrast, are not identical (\$30,506 raw versus \$30,070 isotonic), because the oracle search independently re-optimizes the threshold on final-evaluation data for each score scale, and isotonic's tied plateaus mean the two searches are not guaranteed to land on matching rank positions. With heterogeneous case costs or decisions

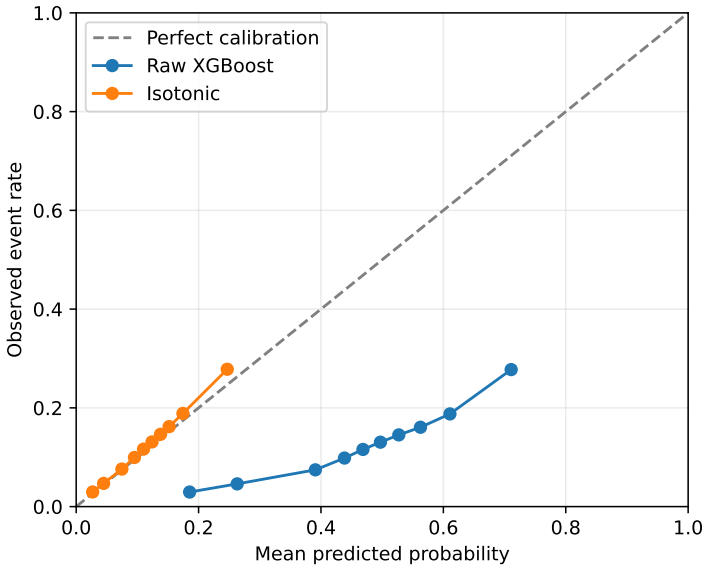

Figure 3: Quantile-binned reliability diagram, full catalog, final-evaluation period only.

that consume probabilities directly, calibration can still affect the action itself; it did not in this single-threshold, constant-cost setting's ex-ante policy specifically.

7.4 Serving artifact versus operating policy

The repository's served artifact is isotonic-calibrated. A calibrated threshold of 0.2801 was also stored to approximately preserve the alert volume of the historical raw 0.80 cutoff. That is a workload-matching policy, not the cost-selected policy evaluated above, and the present experiments do not establish it as economically optimal. Separating the artifact from the policy resolves the apparent contradiction between "deploy calibrated probabilities" and "choose an operating point from explicit costs or capacity."

8 Model-Family Robustness

The results in Sections 6 and 7 use XGBoost [Chen and Guestrin, 2016] throughout, selected at the start of this project without being tested against alternatives. An earlier version of this comparison reported a single 100-item run with thresholds selected and evaluated on the same holdout – an oracle-style estimate presented as if it were a reliable conclusion, and inconsistent with the three-way chronological split used everywhere else in this paper. We rerun it here under the same standard as Section 7: identical train/calibration/final-evaluation periods, the corrected item-store label, three seeds varying each model's own bagging randomness, complexity matched by early stopping on the calibration period rather than forcing identical hyperparameters across libraries with different regularization behavior, and the same oracle-versus-ex-ante threshold separation used throughout.

We compare XGBoost against LightGBM [Ke et al., 2017], CatBoost [Prokhorenkova et al., 2018], and a logistic regression baseline, at the 1,000-series scale. Following Section 7's convention, Brier Skill Score is reported on isotonic-calibrated scores, since raw class-weighted scores from every boosted-tree model here have deeply negative raw-score BSS despite good ranking (Average Precision) – an expected consequence of severe raw overconfidence, not a defect specific to any one

library, and the same phenomenon already implied by Table 3's own raw-versus-reference Brier gap.

Table 4: Model-family comparison, 1,000-series scale, corrected label, mean [range] across 3 seeds. Brier Skill Score (BSS) is reported after isotonic calibration, matching Section 7's convention.

Model	AP	BSS (calibrated)	Ex-ante net	Oracle net
XGBoost	0.179 [0.166, 0.185]	0.035 [0.028, 0.039]	+$11 [0, 34]	+$48 [0, 134]
CatBoost	0.182 [0.181, 0.183]	0.036 [0.036, 0.037]	−$34 [−110, 20]	+$15 [8, 20]
LightGBM	0.150 [0.147, 0.152]	0.024 [0.022, 0.025]	$0 [0, 0]	$0 [0, 0]
Logistic regression	0.134 [fixed]	0.012 [fixed]	−$14 [fixed]	−$14 [fixed]

Two findings are stable across all three seeds. First, XGBoost and CatBoost are consistently and clearly ahead of both LightGBM and the logistic regression baseline on average precision and calibrated Brier Skill Score; the gap between XGBoost and CatBoost themselves, by contrast, is small enough – and their ranges overlap closely enough – that the two are not meaningfully distinguishable at this scale, and which of the two nominally leads is itself sensitive to the exact software environment (a second run of this same script in a different environment with slightly different library versions reordered them). Second, the economic outcome is close to zero for every model except logistic regression, consistent with the near-breakeven business case already established in Section 6 at this sample size – model choice does not rescue an operating point that is fundamentally close to breakeven under the stated costs. Logistic regression is the one model where the gap is not noise: it is worse on every metric, including at its own oracle (upper-bound) threshold, indicating a genuine ranking-quality shortfall rather than sampling variance. Logistic regression's metrics do not vary across seeds because the `lbfgs` solver used here is deterministic given fixed data; `random_state` affects only the three boosted-tree models' bagging behavior.

One additional nuance is worth reporting because it complicates a simple "one model wins" narrative: LightGBM's *raw*, uncalibrated Brier score (0.084 across all three seeds) sits almost exactly at the prevalence-only reference (0.084), whereas XGBoost's and CatBoost's raw Brier scores (0.23–0.24 for both) are roughly 2.7 times worse than that same reference before calibration. LightGBM is, in this specific configuration, substantially better calibrated *out of the box*, at the cost of weaker ranking. This is consistent with – and does not contradict – the paper's broader point in Section 7: ranking quality and calibration quality are separable properties of a model, and a library's default behavior on one does not predict its behavior on the other.

We conclude that XGBoost's original selection is reasonably well-supported on ranking and calibrated-probability grounds specifically against LightGBM, and tied with CatBoost, rather than uniquely justified against every alternative. The near-zero, noise-dominated economic differences among XGBoost, CatBoost, and LightGBM mean this paper's central claims about business impact (Section 6) and calibration (Section 7) are not an artifact of the specific gradient-boosting library chosen.

9 Discussion

The unifying theme is that each finding required testing a specific mechanism rather than trusting a plausible aggregate metric. Chronological holdout was necessary but did not reveal future-informed label construction, a contemporaneous rolling feature, the proxy's grouping choice, or probability miscalibration.

A grouping choice in a label definition is an estimand choice, not an implementation detail. Probability calibration and threshold selection are also distinct: calibration governs the meaning

of scores; threshold selection governs action under stated costs or capacity. The appropriate Brier reference is essential because a large before/after improvement can coexist with only modest skill over prevalence. Section 8's model-family comparison shows these conclusions are not an artifact of the specific boosting library used, though it also shows that "the best model" is itself a comparison whose exact ranking is sensitive to the software environment: XGBoost and CatBoost tie on ranking and calibrated probability quality across two different environments in which this comparison was run, while LightGBM is consistently better calibrated in its raw, uncalibrated state.

10 Limitations

The label is not a verified stockout or disruption outcome; M5 contains no inventory, fulfillment, or lost-sales field. The item-store revision is appropriate only for the stated series-relative estimand. Correlations do not prove causality, the store-level interval is wide because $n = 10$, and the small negative within-item residual correlation after normalization is statistically detectable largely because $n = 30{,}490$. Discrete, zero-inflated sales also produce unequal event rates despite a common percentile.

The model is evaluated on later observations from the same item-store series, not on unseen items, stores, or retailers. Research decisions were informed by earlier examinations of the original holdout, so this remains a retrospective case study rather than a preregistered confirmatory experiment. Full-catalog training uses a 20% row sample of the training period. The cost values are illustrative, and 5,000-series results vary materially by seed. Threshold selection on the calibration period avoids final-period tuning, but the same calibration period also fits isotonic regression; a four-way split would provide an additional policy-selection set. Low recall at economically positive operating points makes the system a queue-prioritization aid, not autonomous detection.

Reproduction uses Python 3.12.13, NumPy 2.4.6, pandas 2.3.3, scikit-learn 1.9.0, and XGBoost 3.3.0. The commands and machine-readable outputs are in `scripts/paper_audit.py`, `scripts/train_calibrate_full_catalog.py`, `scripts/model_comparison_robust.py`, and `paper_draft/genera`
Code is available at https://github.com/suhailyunus/supply-chain-stress-prediction; an immutable release should be archived for submission.

11 Conclusion

This case study shows that a proxy label can be predictable and still be misaligned with its intended meaning. Testing grouping choices exposed a strong volume association, temporal regression tests exposed two leakage mechanisms, and a proper prevalence reference tempered an initially overstated calibration claim. Isotonic calibration repaired overconfidence but added only modest Brier skill over prevalence. Under one global threshold and constant costs, calibration changed probability meaning far more than decision value, while a model-family robustness check confirmed these conclusions hold across gradient-boosting libraries. The broader lesson is procedural: validate the target estimand, the information set, probability meaning, and operating policy as separate objects.

References

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):Article 15.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154.

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364.

Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632.

Platt, J. (1999). Probabilistic outputs for support vector machines and comparison to regularized likelihood methods. *Advances in Large Margin Classifiers*, pages 61–74.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems 31*, pages 6638–6648.

Syntetos, A. A., Boylan, J. E., and Croston, J. D. (2005). On the categorization of demand patterns. *Journal of the Operational Research Society*, 56(5):495–503.

Zadrozny, B. and Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 694–699.